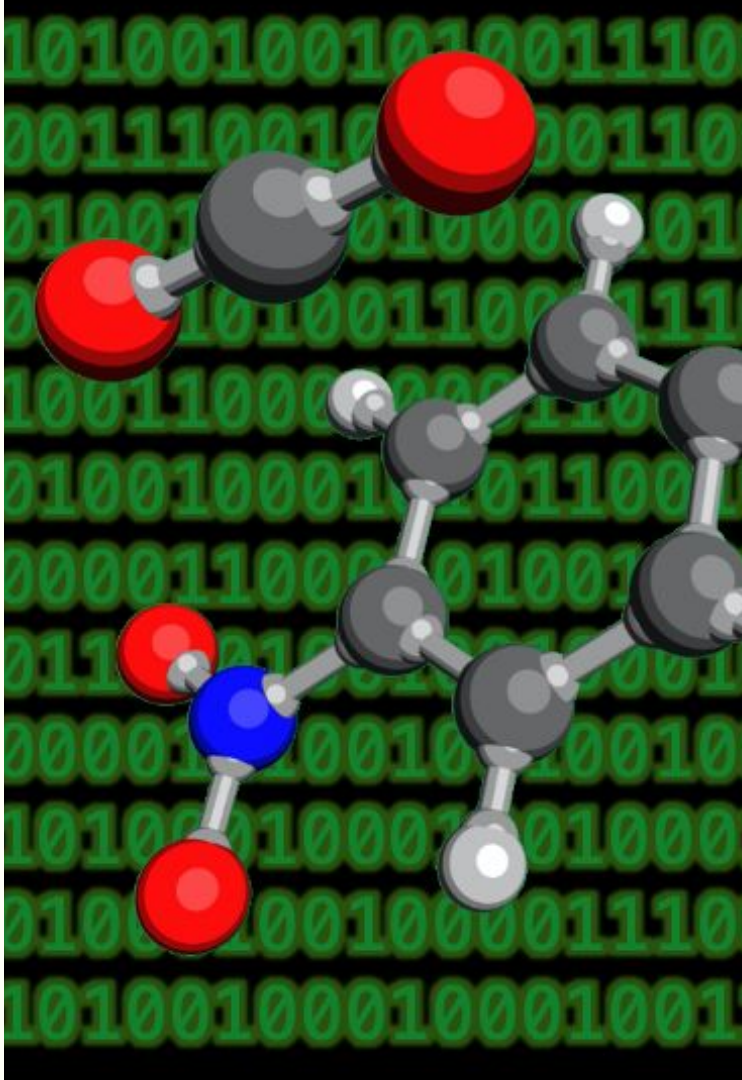
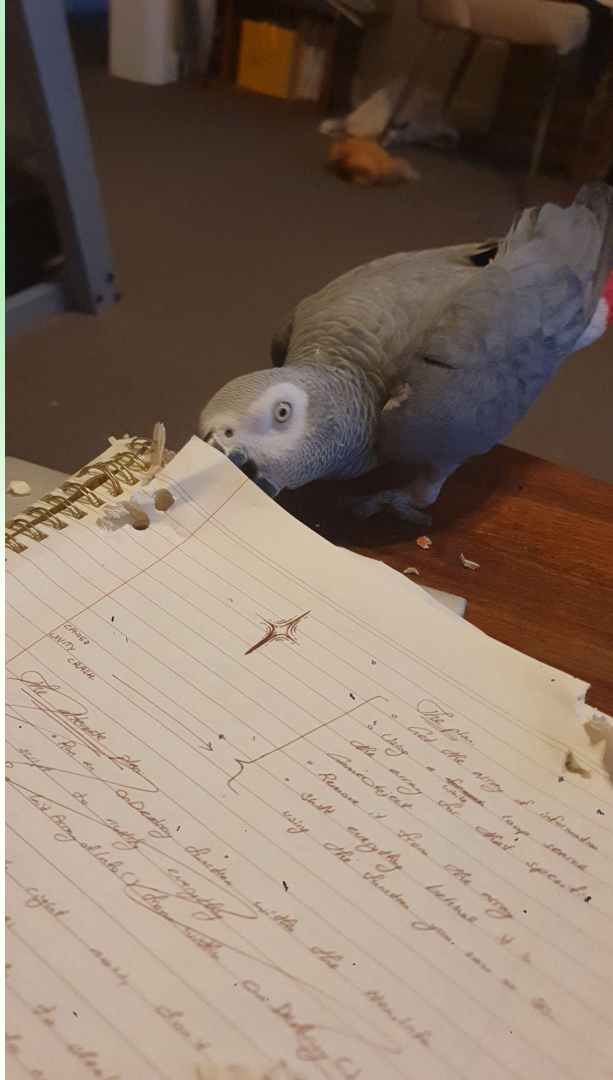
COMP1521
Wk1 Tutorial

About Me

- Sophia Budkin
- 2nd year Comp Sci/Chemistry student
- Interests in lower-level programming + Computational Chemistry
- Icebreaker fact...



I Have a Bird!



Icebreaker Time!

Programs In Memory

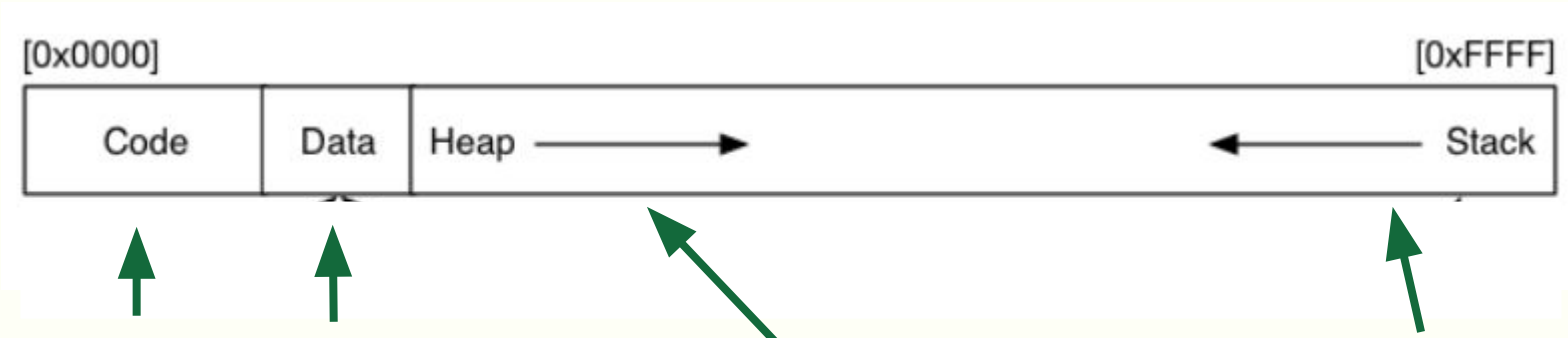
What’s in an Executable File?

A **LOT** of information, structured into subcategories

- .text: stores the translated lines of code
- .data: stores any global variables/arrays, as well as things like string literals.
- A lot of other information that is not necessarily to know for the course!

What Happens When We Run a Program?

- The memory the running program occupies is in the RAM (rapid-access memory)



.text section
Programming instructions
Loaded once from executable file

.data section (global vars + strings)
Loaded once from executable file, further modified during runtime

Heap
User-allocated memory (malloc/free)
Set up by computer when running

Stack
Local variables, function arguments, basically anything used inside a function.
Set up by the computer when running, automatically cleaned up

Q2 - Local Vars, Global Vars and Strings In Data

```c
#include <stdio.h>

char *s1 = "abc";

int main(void) {
  char *s2 = "def";
  // ...
}
```

What is the difference between s1 and s2, and where they are located in memory?

What about the strings “abc” and “def” – where are they stored in memory?

(solutions next slide or on the tutorial answers page)

Recursion

Factorials in Maths

Option 1: Iterative

$n! = 1 \times 2 \times 3 \times \ldots \times n$

Option 2: Recursive

General case: $n! = n \times (n - 1)!$

Base case: $1! = 1$

Called recursive since definition relies on “calling” itself.

Coding A Factorial-Computing Function

Option 1: Iterative

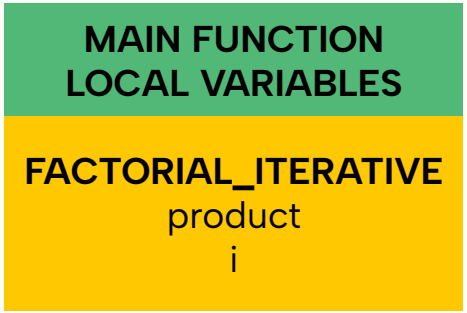
```
// Calculates factorial iteratively, i.e.
// using the method n! = 1 * 2 * ... * n
int factorial_iterative(int num) {
    int product = 1;

    for (int i = 2; i <= num; i++) {
        product *= i;
    }

    return product;

}
```

Option 2: Recursive

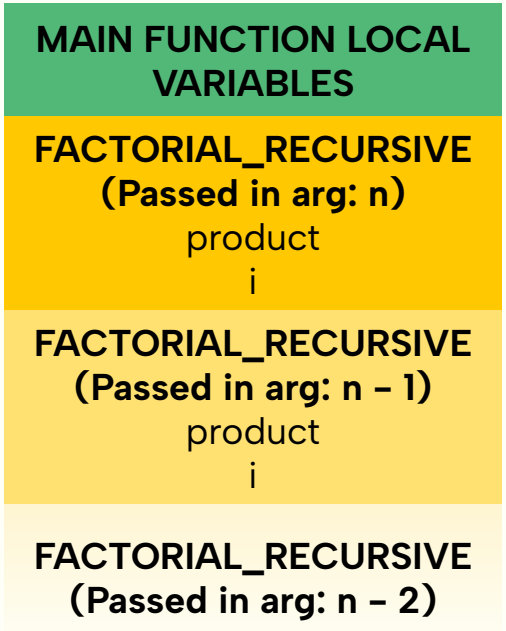
```
// Calculates factorial recursively, i.e.
// using the general method of n! = n *
// (n - 1)! and the base case of 1! = 1.
int factorial_recursive(int num) {
    // Base case - must be handled separately!
    if (num == 1) {
        return 1;
    }

    // General case
    return num * factorial_recursive(num - 1);
}
```

Under The Hood: What Does the Memory Look Like for Both Approaches?

Option 1: Iterative



Option 2: Recursive



Q7 - Recursive Implementation of Sum

```
int sum(int n) {
    int result = 0;
    for (int i = 0; i <= n; i++) {
        result += i;
    }
    return result;
}
```

How could this be rewritten using recursion instead?

Things to focus on when planning:

- What would a base/stopping case be?
- How could you make a general case that recursively calls itself?
- What would the difference in the program stack be between the two implementations?

(solution code versions available on GitHub or tutorial question answers)

The Manual

Useful Sections (1/4)

Function Used as Example: printf

```
SYNOPSIS
       #include <stdio.h>

       int printf(const char *restrict format, ...);
```

Use for function prototype + double-checking necessary libraries to include!

Useful Sections (2/4)

DESCRIPTION

The functions in the **printf**() family produce output according to a format as described below. The functions **printf**() and **vprintf**() write output to stdout, the standard output stream; **fprintf**() and **vfprintf**() write output to the given output stream; **sprintf**(), **snprintf**(), **vsprintf**(), and **vsnprintf**() write to the character string str.

Details the function's behaviour – very helpful if you forget the details/unsure of how to use a specific function!

Useful Sections (3/4)

```
RETURN VALUE
       Upon  successful  return,  these  functions  return  the number of bytes
       printed (excluding the null byte used to end output to strings).

       The functions snprintf() and vsnprintf() do not  write  more  than  size
       bytes  (including  the terminating null byte ('\0')).  If the output was
       truncated due to this limit, then the return  value  is  the  number  of
       characters  (excluding  the terminating null byte) which would have been
       written to the final string if enough space had been available.  Thus, a
       return value of size or more means that the output was truncated.   (See
       also below under CAVEATS.)

       If an output error is encountered, a negative value is returned.
```

Details the return value of the function under normal and error conditions – very good to reference when setting up error checking!

Useful Sections (4/4)

Function Used as Example: fopen (you don’t need to understand how it works yet!)

ERRORS

EINVAL The mode provided to **fopen**(), **fdopen**(), or **freopen**() was invalid.

The **fopen**(), **fdopen**(), and **freopen**() functions may also fail and set errno for any of the errors specified for the routine **malloc**(3).

The **fopen**() function may also fail and set errno for any of the errors specified for the routine **open**(2).

The **fdopen**() function may also fail and set errno for any of the errors specified for the routine **fcntl**(2).

The **freopen**() function may also fail and set errno for any of the errors specified for the routines **open**(2), **fclose**(3), and **fflush**(3).

Certain functions have a lot of reasons why they could fail + special ways of indicating what different errors may have occurred – provides a comprehensive list of why a function call failed! (Mainly relevant for file handling portion of course)

Q5 - Practicing Using the Manual

5. Write a c program `count_chars.c` that uses getchar to read in characters until the user enters Ctrl-D and then prints the total number of characters entered.

   Use `man 3 getchar` to look at the manual entry.

Hint: what function that you are already familiar with from 1511 is getchar() very similar to? What are the syntax differences when using it?

(solutions on the GitHub and on the tutorial answer page)

Note: This is very complicated, and we will not diving into the nitty-gritty. Do not worry, we do NOT expect at all a detailed knowledge of how it works!

Compilation Steps: Overview

1. Preprocessing

All code with hashtags (e.g #include statements and #defines) is modified to remove them:

- #include statements: the contents of included files are just pasted in at the top (e.g #include <stdio.h> → all function definitions and constants from stdio.h are pasted in the top of our file)
- #define statements: all instances of the constant in the file are replaced with whatever the value of the constant was
- All comments removed

Example on GitHub: preprocessed_q10.i (sample screenshot below)

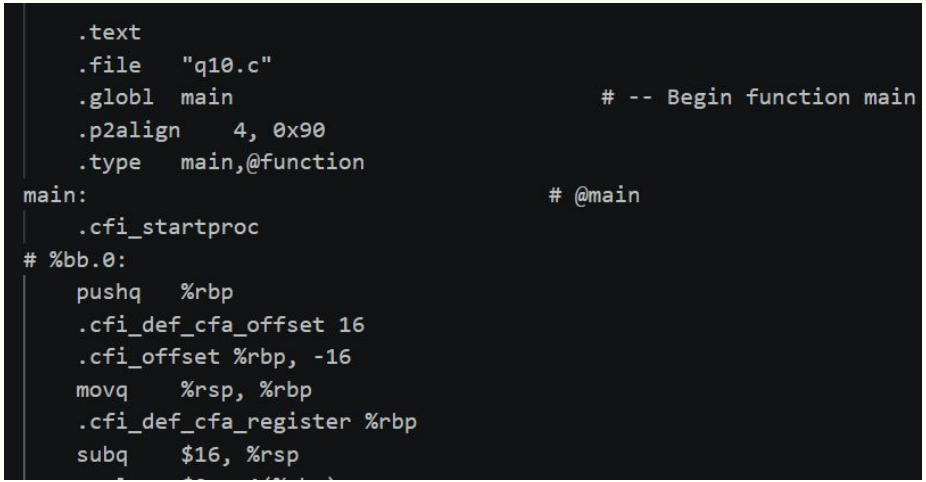
```c
// Original starting C code.

#include <stdio.h>
#define N 10

int main(void) {
    char str[N] = { 'H', 'i', '\0' };
    printf("%s\n", str);
    return 0;
}
```

```c
extern int __uflow (FILE *);
extern int __overflow (FILE *, int);
# 2 "q10.c" 2


int main(void) {
    char str[10] = { 'H', 'i', '\0' };
    printf("%s\n", str);
    return 0;
}
```

2. Compilation to Assembly Code

Preprocessed code is converted into assembly code that's specific to whatever CPU your computer has

- Assembly code is specific to the structure of the CPU it's designed for



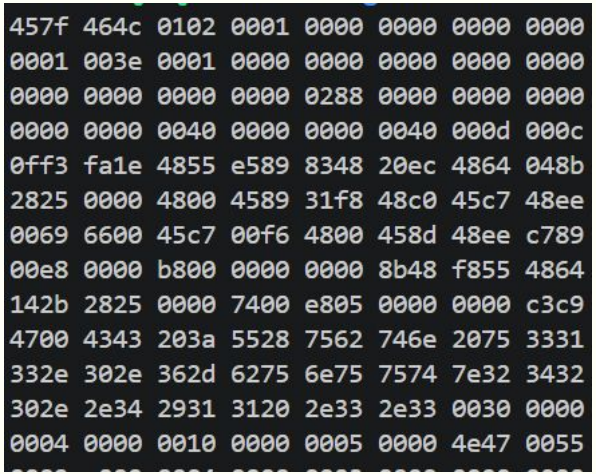
```
    .text
    .file   "q10.c"
    .globl  main                            # -- Begin function main
    .p2align    4, 0x90
    .type   main,@function
main:                                   # @main
    .cfi_startproc
# %bb.0:
    pushq   %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset %rbp, -16
    movq    %rsp, %rbp
    .cfi_def_cfa_register %rbp
    subq    $16, %rsp
```

3. Assembly (Conversion of Assembly Code to Machine Code)

Assembly code for the file is converted into machine code

- NOTE: functions referenced from other files (e.g printf in this case) don't yet have their machine code linked together with our main file!
- Known as "relocatable object files" at this stage
- Not really human-readable anymore

Example on github: q10.o (screenshot shows some of the contents in hexadecimal)



4. Linking

Machine code for several files that reference each other (e.g machine code for q10.c and for printf) are linked together into one executable that can actually run!

Example on github: q10 (screenshot shows some of the contents in hexadecimal)